

product page you set product to true:

```
import requests

resp = requests.post(
    "https://api.zyte.com/v1/extract",
    auth=("YOUR_ZYTE_API_KEY", ""),
    json={
        "url": "https://books.toscrape.
com/catalogue/a-light-in-the-attic_1000/
index.html",
        "product": True,
    },
)
product = resp.json()["product"]
print(product["name"],
product["price"], product["currency"],
product["availability"])
```

You do not write a single selector. The response carries a product object with fields already named and typed.

Running the snippet above prints A Light in the Attic 51.77 GBP InStock, and the full object includes currencyRaw as the original £ string, a sku, a brand with its own name, a breadcrumbs array walking Home, Books, Poetry, the mainImage.url, a description, and a metadata block. That metadata even reports an extraction probability, which was 0.997 on this page, so you can flag low-confidence results instead of trusting everything blindly. Product is not the only type available. The same pattern covers productList, article and articleList, jobPosting, forumThread, and search results through serp, each returning its own structured shape. Figure 2 shows the real response for the book above, pretty-printed in a terminal.

The reason this matters goes back to the third wall. When extraction is a

model asking “what is the product on this page” rather than a selector asking “what is inside div.price_color,” a layout change on the target site no longer breaks your parser. That is the difference between maintaining a scraper and running one.

Scrapy: The open source engine for web scraping at scale

A single API call is perfect for one page or a handful. The moment you want a whole catalogue, though, you need something to drive the crawl: to follow links, manage concurrency, retry failures, throttle politely, and push clean records out the other end. That is exactly what Scrapy is for, and it has been the workhorse of Python web scraping for well over a decade. It is fully open source and is developed and maintained by Zyte, the same one behind the API from the previous section.

```
zsh — Zyte API automatic extraction

$ curl -s -u $ZYTE_API_KEY: --header 'Content-Type: application/json' \
  --data '{"url": ".../a-light-in-the-attic_1000/", "product": true}' \
  https://api.zyte.com/v1/extract | python -m json.tool
{
  "url": "https://books.toscrape.com/catalogue/a-light-in-the-attic_1000/index.html",
  "statusCode": 200,
  "product": {
    "name": "A Light in the Attic",
    "price": "51.77",
    "currency": "GBP",
    "currencyRaw": "£",
    "availability": "InStock",
    "sku": "a897fe39b1053632",
    "brand": { "name": "Books to Scrape" },
    "breadcrumbs": [
      { "name": "Home" }, { "name": "Books" },
      { "name": "Poetry" }, { "name": "A Light in the Attic" }
    ],
    "mainImage": { "url": "https://books.toscrape.com/media/.../fe72...jpg" },
    "description": "It's hard to imagine a world without A Light in the Attic ...",
    "metadata": { "probability": 0.9976, "dateDownloaded": "2026-07-03T07:41:25Z" }
  }
}
# structured fields, straight from the URL, with no CSS or XPath selectors
```

Figure 2: The response from a single automatic-extraction call. Every field is named and typed, and the metadata block reports an extraction confidence you can check before trusting the record